

Lock-Based Data Structures

Lecture 10

Professor Amanda Bienz

Lock-Based Concurrent Data Structure

- Race conditions happen around shared state
- Good programming practice generally encompasses state in a data structure or object
- Common goal : add locks to a data structure to make the structure thread safe
- Considerations:
 - Correctness : does data structure still do what we want?
 - Scalability : more threads shouldn't slow down operations

Lock-Based Concurrent Data Structure

- Race conditions happen around shared state
- Good programming practice generally encompasses state in a data structure or object
- Common goal : add locks to a data structure to make the structure thread safe
- Considerations:
 - Correctness : does data structure still do what we want?
 - Scalability : more threads shouldn't slow down operations
 - **These two goals are often in conflict**

Example : Concurrent Counters without Locks

- Simple, but not correct with multiple updaters

```
1      typedef struct __counter_t {
2          int value;
3      } counter_t;
4
5      void init(counter_t *c) {
6          c->value = 0;
7      }
8
9      void increment(counter_t *c) {
10         c->value++;
11     }
12
13     void decrement(counter_t *c) {
14         c->value--;
15     }
16
17     int get(counter_t *c) {
18         return c->value;
19     }
```

Example : Concurrent Counters with Locks

- Add a single lock acquired when calling a routine that manipulates the data structure
- Reminder : Pthread_XXX is a wrapper around pthread_XXX with an error check

```
1      typedef struct __counter_t {
2          int value;
3          pthread_lock_t lock;
4      } counter_t;
5
6      void init(counter_t *c) {
7          c->value = 0;
8          Pthread_mutex_init(&c->lock, NULL);
9      }
10
11     void increment(counter_t *c) {
12         Pthread_mutex_lock(&c->lock);
13         c->value++;
14         Pthread_mutex_unlock(&c->lock);
15     }
16
```

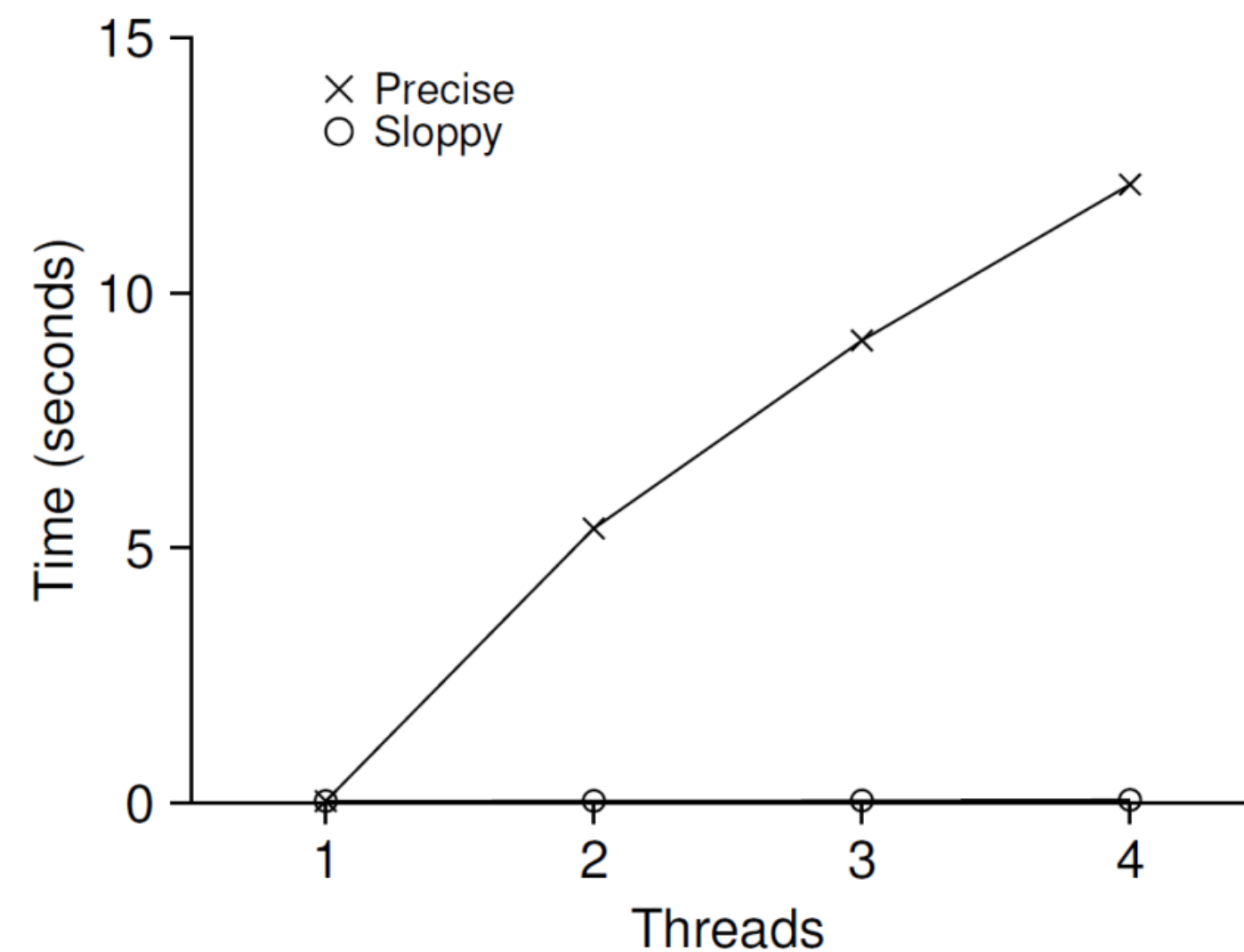
Example : Concurrent Counters with Locks (Cont.)

- Add a single lock acquired when calling a routine that manipulates the data structure
- Reminder : Pthread_XXX is a wrapper around pthread_XXX with an error check

```
(Cont.)
17     void decrement(counter_t *c) {
18         Pthread_mutex_lock(&c->lock);
19         c->value--;
20         Pthread_mutex_unlock(&c->lock);
21     }
22
23     int get(counter_t *c) {
24         Pthread_mutex_lock(&c->lock);
25         int rc = c->value;
26         Pthread_mutex_unlock(&c->lock);
27         return rc;
28     }
```


The performance costs of the simple approach

- Each thread updates a single shared counter
 - Each thread updates the counter one million times
- iMac with four Intel 2.7GHz i5 CPUs



Synchronized counter **scales poorly.**

Goal : Perfect Scaling

- Even though more work is done, it is done in parallel
- The time taken to complete the task is *not increased*
- For the counters example :
 - Perfect scaling with N threads is N times the updates in the same time
 - Our example when from less than 0.1 seconds with 1 thread to about 12 seconds with 4 threads
- Contending on locks and data structures is **very expensive** due to architectural reasons
 - Cross-CPU communication (e.g. for locking)
 - Cross-CPU cache interferences (counter moves between CPU caches, so things that hit with 1 CPU miss all the time with 2)

Sloppy Counter

- A common approach : redefine the problem by relaxing consistency
 - Often don't need perfect count at any given time
 - Don't want to lose any counts (eventually see all increments)
- Sloppy counter works by representing...
 - A single **logical counter** via numerous local physical counters, one per CPU core
 - A single **global counter**
 - There are **locks** : one for each local counter and one for global
- Example : machine with 4 CPUs would have 4 local counters and one global counter

Basic Idea of Sloppy Counting

- When a thread running on a core wishes to increment the counter:
 - It increments its local counter
 - Each CPU has its own local counter
 - Threads across CPUs can update local counters *without contention*
 - Thus, counter updates are scalable
- Local values are periodically transferred to global counter
 - Acquire global lock, increment by local counter's value, reset local counter to zero

Basic Idea of Sloppy Counting (Cont.)

- How often the local-to-global transfer occurs is determined by a threshold S (sloppiness)
 - Smaller S : counter behaves like the non-scalable counter
 - Larger S : more scalable counter, but further off global value might be from actual count
- Note : it is not a counter per thread, just per CPU
 - This is why we have a lock per local counter : multiple threads could update the counter on a single CPU
 - A counter per thread would eliminate this, but result in a lot of states if you have a lot of threads

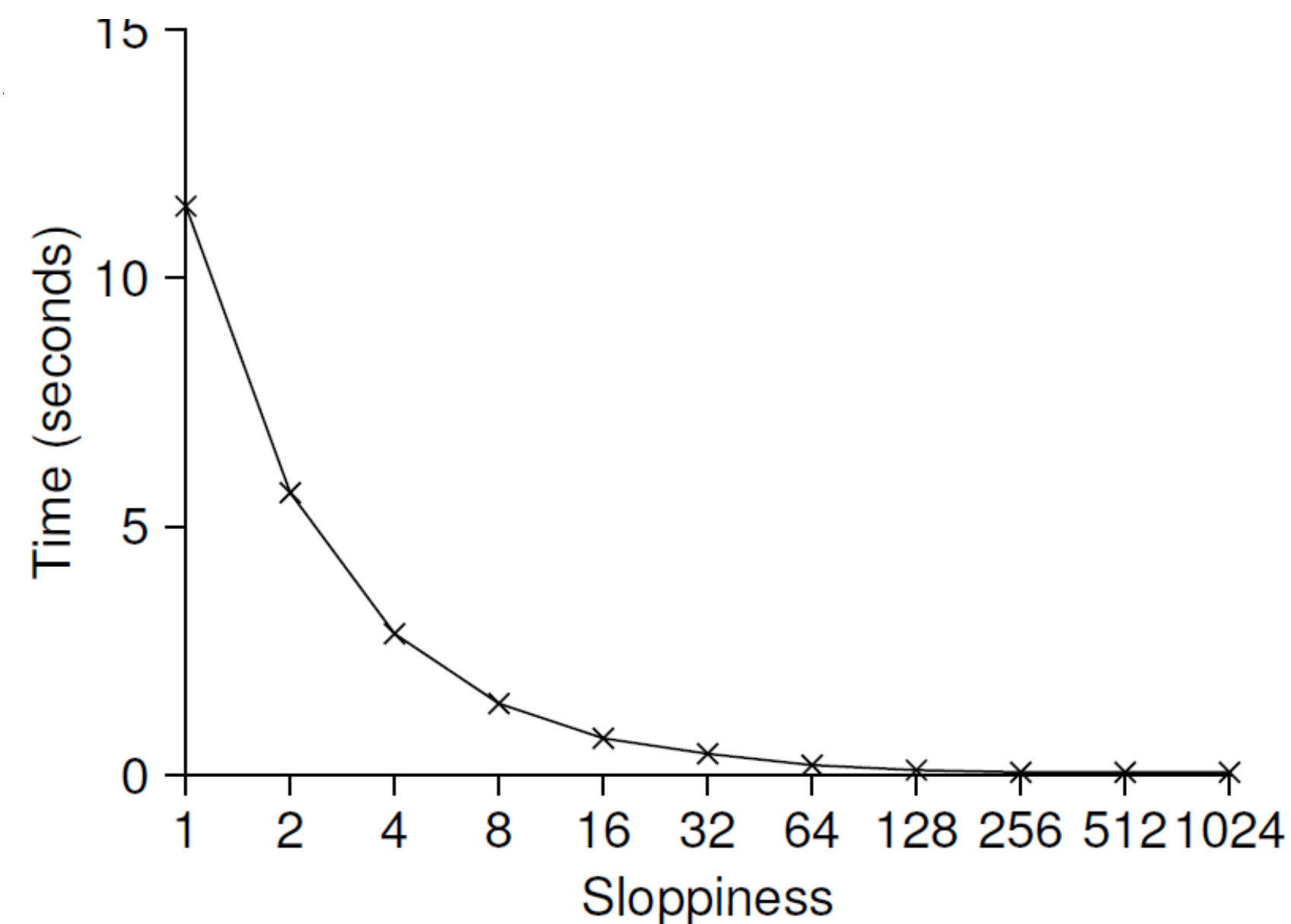
Sloppy Counter Example

- Tracing the Sloppy Counters
 - Threshold S is 5, four CPUs (on thread per CPU)
 - Each thread updates their local counters (L_1 to L_4)

Time	L_1	L_2	L_3	L_4	G
0	0	0	0	0	0
1	0	0	1	1	0
2	1	0	2	1	0
3	2	0	3	1	0
4	3	0	3	2	0
5	4	1	3	3	0
6	5 $\rightarrow$ 0	1	3	4	5 (from L_1)
7	0	2	4	5 $\rightarrow$ 0	10 (from L_4)

Importance of the Threshold Value S

- Each four threads increments a counter 1 million times on four CPUs
- Low S : poor performance, global count always accurate
- High S : better performance, global count is not always accurate



Scaling Sloppy Counters

Sloppy Counter Implementation

```
1  typedef struct __counter_t {
2      int global;           // global count
3      pthread_mutex_t glock; // global lock
4      int local[NUMCPUS];   // local count (per cpu)
5      pthread_mutex_t llock[NUMCPUS]; // ... and locks
6      int threshold;        // update frequency
7  } counter_t;
8
9  // init: record threshold, init locks, init values
10 //      of all local counts and global count
11 void init(counter_t *c, int threshold) {
12     c->threshold = threshold;
13
14     c->global = 0;
15     pthread_mutex_init(&c->glock, NULL);
16
17     int i;
18     for (i = 0; i < NUMCPUS; i++) {
19         c->local[i] = 0;
20         pthread_mutex_init(&c->llock[i], NULL);
21     }
22 }
23
```


Sloppy Counter Implementation

```
(Cont.)
24 // update: usually, just grab local lock and update local amount
25 //           once local count has risen by 'threshold', grab global
26 //           lock and transfer local values to it
27 void update(counter_t *c, int threadID, int amt) {
28     pthread_mutex_lock(&c->llock[threadID]);
29     c->local[threadID] += amt; // assumes amt > 0
30     if (c->local[threadID] >= c->threshold) { // transfer to global
31         pthread_mutex_lock(&c->glock);
32         c->global += c->local[threadID];
33         pthread_mutex_unlock(&c->glock);
34         c->local[threadID] = 0;
35     }
36     pthread_mutex_unlock(&c->llock[threadID]);
37 }
38
39 // get: just return global amount (which may not be perfect)
40 int get(counter_t *c) {
41     pthread_mutex_lock(&c->glock);
42     int val = c->global;
43     pthread_mutex_unlock(&c->glock);
44     return val; // only approximate!
45 }
```


Concurrent Linked Lists

```
1      // basic node structure
2      typedef struct __node_t {
3          int key;
4          struct __node_t *next;
5      } node_t;
6
7      // basic list structure (one used per list)
8      typedef struct __list_t {
9          node_t *head;
10         pthread_mutex_t lock;
11     } list_t;
12
13     void List_Init(list_t *L) {
14         L->head = NULL;
15         pthread_mutex_init(&L->lock, NULL);
16     }
17
```

(Cont.)

Concurrent Linked Lists (Cont.)

(Cont.)

```
18     int List_Insert(list_t *L, int key) {
19         pthread_mutex_lock(&L->lock);
20         node_t *new = malloc(sizeof(node_t));
21         if (new == NULL) {
22             perror("malloc");
23             pthread_mutex_unlock(&L->lock);
24             return -1; // fail
25         }
26         new->key = key;
27         new->next = L->head;
28         L->head = new;
29         pthread_mutex_unlock(&L->lock);
30         return 0; // success
31     }
```

(Cont.)

Concurrent Linked Lists (Cont.)

(Cont.)

```
32
32     int List_Lookup(list_t *L, int key) {
33         pthread_mutex_lock(&L->lock);
34         node_t *curr = L->head;
35         while (curr) {
36             if (curr->key == key) {
37                 pthread_mutex_unlock(&L->lock);
38                 return 0; // success
39             }
40             curr = curr->next;
41         }
42         pthread_mutex_unlock(&L->lock);
43         return -1; // failure
44     }
```

Concurrent Linked Lists (Cont.)

- The code acquires a lock in the insert routine upon entry
- The code releases the lock upon exit
 - If malloc() happens to fail, the code must also release the lock before failing the insert
 - This kind of exceptional control flow has been shown to be quite error prone
 - You have to release a single lock in multiple places
 - Changes to how you lock/unlock have to propagate to multiple places in the code (and it is easy to miss one)
- **Solution : the lock and release only surround the actual critical section in the insert code**

Concurrent Linked List Insert : Rewritten

```
1      void List_Init(list_t *L) {
2          L->head = NULL;
3          pthread_mutex_init(&L->lock, NULL);
4      }
5
6      void List_Insert(list_t *L, int key) {
7          // synchronization not needed
8          node_t *new = malloc(sizeof(node_t));
9          if (new == NULL) {
10             perror("malloc");
11             return;
12         }
13         new->key = key;
14
15         // just lock critical section
16         pthread_mutex_lock(&L->lock);
17         new->next = L->head;
18         L->head = new;
19         pthread_mutex_unlock(&L->lock);
20     }
21
```

Scaling Linked List

- Current linked list has poor scalability : lock the entire list while you walk it
- Hand-over-hand locking (lock coupling)
 - Add a lock per node of the list instead of having a single lock for the entire list
 - While traversing the list: 1. Grab next node's lock and 2. Release current node's lock
 - Enable a high degree of concurrency in list operations
 - However, in practice, the overheads of acquiring and releasing locks for each node of a list traversal is prohibitive
- **Scaling arbitrary linked lists is difficult because the sheer amount of state to be protected**

Michael and Scott Concurrent Queues

- What if all we want is a queue?
- There are two locks
 - One for the **head** of the queue
 - One for the **tail**
 - **The goal of these two locks is to enable concurrency of enqueue and dequeue operations**
- Add a dummy node
 - Allocated in the queue initialization code
 - Enable the separation of head and tail operations

Concurrent Queues (Cont.)

```
1     typedef struct __node_t {
2         int value;
3         struct __node_t *next;
4     } node_t;
5
6     typedef struct __queue_t {
7         node_t *head;
8         node_t *tail;
9         pthread_mutex_t headLock;
10        pthread_mutex_t tailLock;
11    } queue_t;
12
13    void Queue_Init(queue_t *q) {
14        node_t *tmp = malloc(sizeof(node_t));
15        tmp->next = NULL;
16        q->head = q->tail = tmp;
17        pthread_mutex_init(&q->headLock, NULL);
18        pthread_mutex_init(&q->tailLock, NULL);
19    }
20
21    (Cont.)
```

Concurrent Queues (Cont.)

(Cont.)

```
21     void Queue_Enqueue(queue_t *q, int value) {
22         node_t *tmp = malloc(sizeof(node_t));
23         assert(tmp != NULL);
24
25         tmp->value = value;
26         tmp->next = NULL;
27
28         pthread_mutex_lock(&q->tailLock);
29         q->tail->next = tmp;
30         q->tail = tmp;
31         pthread_mutex_unlock(&q->tailLock);
32     }
```

(Cont.)

Concurrent Queues (Cont.)

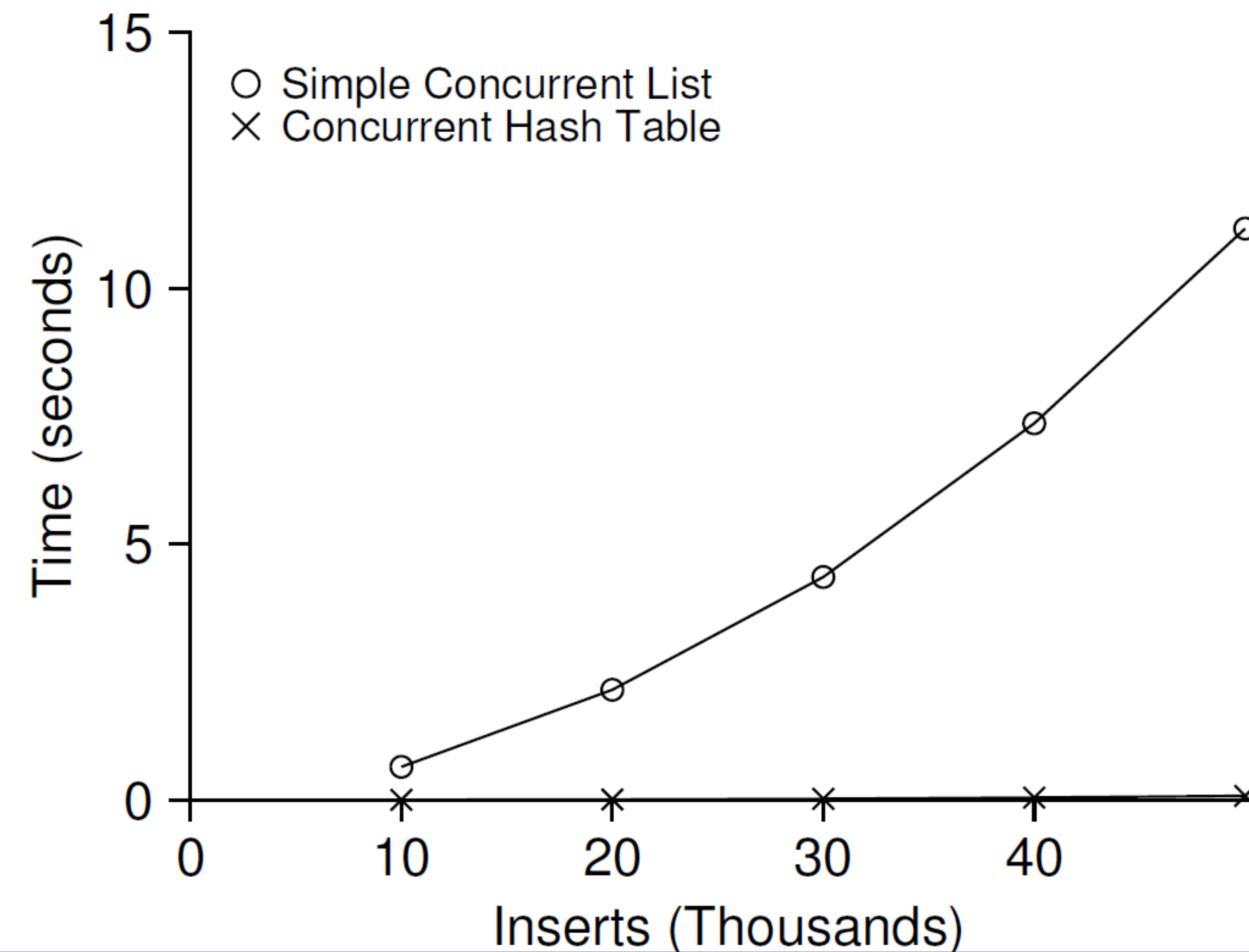
```
(Cont.)
33     int Queue_Dequeue(queue_t *q, int *value) {
34         pthread_mutex_lock(&q->headLock);
35         node_t *tmp = q->head;
36         node_t *newHead = tmp->next;
37         if (newHead == NULL) {
38             pthread_mutex_unlock(&q->headLock);
39             return -1; // queue was empty
40         }
41         *value = newHead->value;
42         q->head = newHead;
43         pthread_mutex_unlock(&q->headLock);
44         free(tmp);
45         return 0;
46     }
```

Concurrent Hash Table

- Focus on a simple hash table
 - The hash table does not resize
 - Build using the concurrent lists
 - It uses a **lock per hash bucket** each of which is represented by a list

Performance of Concurrent Hash Table

- From 10,000 to 50,000 concurrent updates from each of four threads
 - iMac with four Intel 2.7 GHz i5 CPUs



- The simple concurrent hash table **scales magnificently**.
- With few threads $\ll$ buckets, threads are generally on independent lists!

Concurrent Hash Table

```
1      #define BUCKETS (101)
2
3      typedef struct __hash_t {
4          list_t lists[BUCKETS];
5      } hash_t;
6
7      void Hash_Init(hash_t *H) {
8          int i;
9          for (i = 0; i < BUCKETS; i++) {
10             List_Init(&H->lists[i]);
11          }
12      }
13
14      int Hash_Insert(hash_t *H, int key) {
15          int bucket = key % BUCKETS;
16          return List_Insert(&H->lists[bucket], key);
17      }
18
19      int Hash_Lookup(hash_t *H, int key) {
20          int bucket = key % BUCKETS;
21          return List_Lookup(&H->lists[bucket], key);
22      }
```